torch.nn.factory_kwargs#

https://pytorch.org/docs/stable/generated/torch.nn.factory_kwargs.html#torch

Description:

Return a canonicalized dict of factory kwargs. Given kwargs, returns a canonicalized dict of factory kwargs that can be directly passed to factory functions like torch.empty, or errors if unrecognized kwargs are present. This function makes it simple to write code like this: Why should you use this function instead of just passing kwargs along directly? 1. This function does error validation, so if there are unexpected kwargs we will immediately report an error, instead of deferring it to the factory call 2. This function supports a special factory_kwargs argument, which can be used to explicitly specify a kwarg to be used for factory functions, in the event one of the factory kwargs conflicts with an already existing argument in the signature (e.g. in the signature def f(dtype, **kwargs), you can specify dtype for factory functions, as distinct from the dtype argument, by saying f(dtype1, factory_kwargs={"dtype": dtype2}))

Function Signature

```
torch.nn.factory_kwargs(kwargs)[source]#
```

Code Examples

```
class MyModule(nn.Module):
    def __init__(self, **kwargs):
        factory_kwargs = torch.nn.factory_kwargs(kwargs)
        self.weight = Parameter(torch.empty(10,
**factory_kwargs))
```

Detailed Documentation

Documentation

Return a canonicalized dict of factory kwargs.

Given kwargs, returns a canonicalized dict of factory kwargs that can be directly passed to factory functions like torch.empty, or errors if unrecognized kwargs are present.

This function makes it simple to write code like this:

Why should you use this function instead of just passing kwargs along directly?

1. This function does error validation, so if there are unexpected kwargs we will immediately report an error, instead of deferring it to the factory call 2. This function supports a special factory_kwargs argument, which can be used to explicitly specify a kwarg to be used for factory functions, in the event one of the factory kwargs conflicts with an already existing argument in the signature (e.g. in the signature def f(dtype, **kwargs), you can specify dtype for factory functions, as distinct from the dtype argument, by saying f(dtype1, factory_kwargs={"dtype": dtype2}))